1. Construct an AVL Tree by inserting the following keys:
50, 25, 12, 80, 90, 37, 45, 18, 19, 6

An AVL Tree is a self balancing binary search tree in which the height difference (balance factor) of left and right subtree of every node is at most 1.

Balance factor (BF) = Height (left subtree) − Height (right subtree)

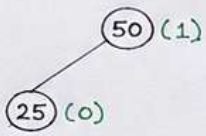
BF ∈ {−1, 0, +1} for every node.

Step by step insertion with rotations:

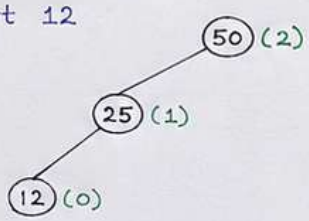
1) Insert 50



2) Insert 25

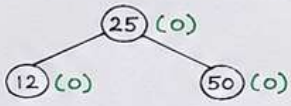


3) Insert 12

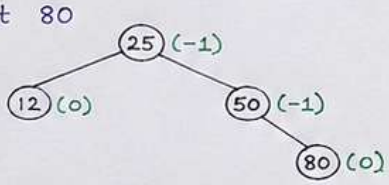


Unbalanced at 50 (BF = +2)
This is Left Left (LL) case
Right Rotation at 50

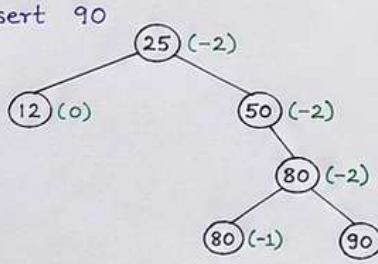
⇒



4) Insert 80



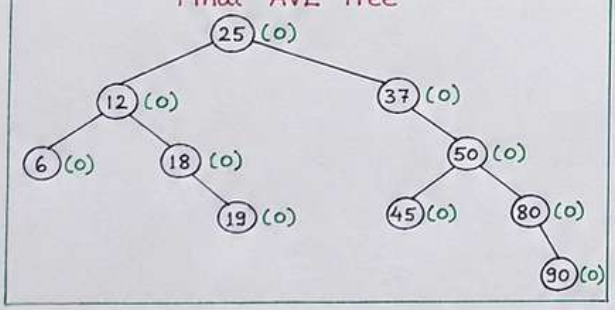
5) Insert 90



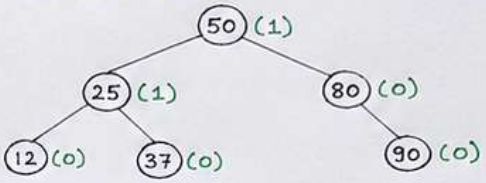
Unbalanced at 25 (BF = −2)
This is Right Right (RR) case
Left Rotation at 25

10) Insert 6

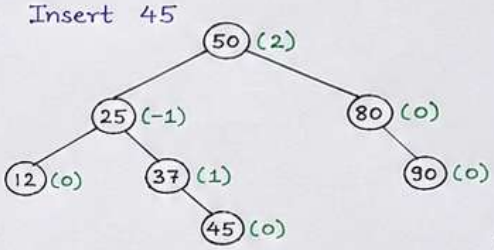
Final AVL Tree



6) Insert 37

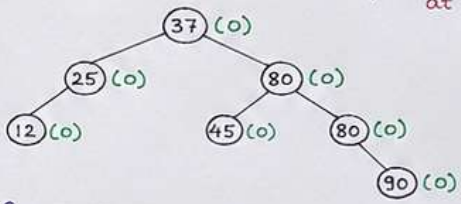


7) Insert 45

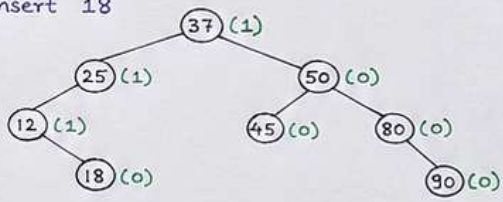


Unbalanced at 50 (BF = +2)
This is Left Right (LR) case
Left Rotation at 25
Right Rotation at 50

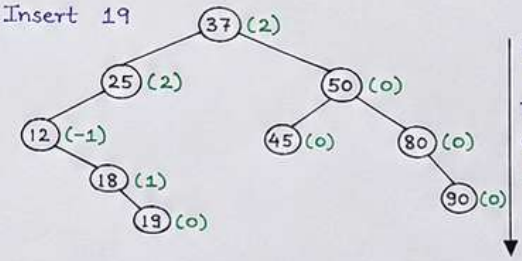
⇒



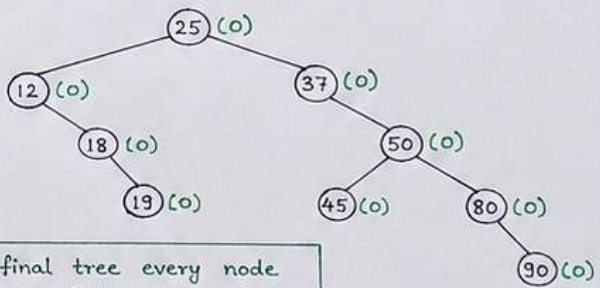
8) Insert 18



9) Insert 19



Unbalanced at 37 (BF = +2)
This is Left Left (LL) case
Right Rotation at 37



In the final tree every node has balance factor −1, 0 or +1. Hence it is a valid AVL Tree.

2. Explain the four types of AVL Tree rotations with suitable examples. Discuss the applications of AVL Trees. (10 Marks)

AVL Tree Rotations:

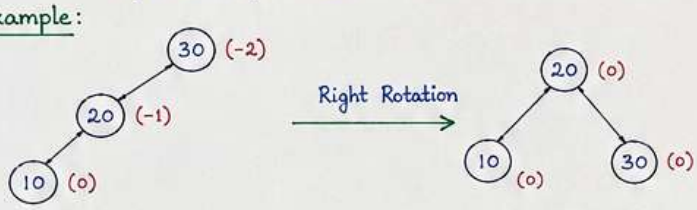
Rotations are used to restore the balance of an AVL Tree when the balance factor of a node becomes -2 or $+2$ due to insertion or deletion.

There are four types of rotations.

1 LL (Left Left) Case – Right Rotation

Condition: The node is unbalanced to the Left and the new key is inserted in the left subtree of the left child.

Example:

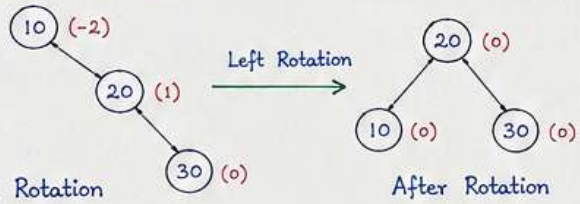


Before Rotation — After Rotation

2 RR (Right Right) Case – Left Rotation

Condition: The node is unbalanced to the right and the new key is inserted in the right subtree of the right child.

Example:

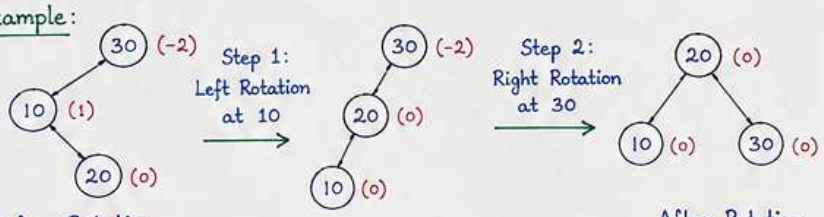


Before Rotation — After Rotation

3 LR (Left Right) Case – Left Rotation + Right Rotation

Condition: The node is unbalanced to the left but the new key is inserted in the right subtree of the left child.

Example:

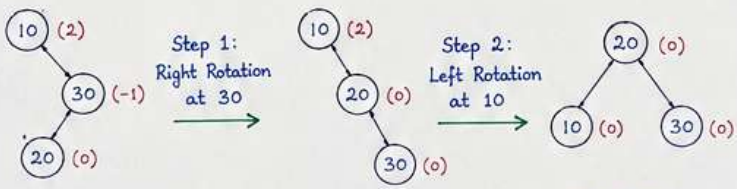


Before Rotation — After Rotation

4 RL (Right Left) Case – Right Rotation + Left Rotation

Condition: The node is unbalanced to the right but the new key is inserted in the left subtree of the right child.

Example:



Before Rotation — After Rotation

Note: After each rotation, the heights and balance factors of the affected nodes are updated.

Balance Factor (BF) = Height(Left Subtree) – Height(Right Subtree)

$BF \in \{-1, 0, +1\}$

Applications of AVL Trees:

AVL Trees are self-balancing BSTs. They ensure that operations remain efficient even in the worst case.

1 Database Systems:
- Used in indexing to speed up search, insert and delete operations.
- Example: Indexing in SQL databases to quickly locate records.

2 File Systems:
- AVL Trees are used in file directories and symbol tables for fast access.
- They help maintain sorted order of file names or keys.

3 Compilers and Interpreters:
- Used in symbol tables to store identifiers (variables, functions etc.) in sorted order.
- Provides fast search, insertion and deletion of symbols.

4 Memory Management:
- Used to manage free memory blocks. AVL Trees help in allocating and deallocating memory efficiently.

5 Network Routing:
- Used in routing tables to store IP addresses or network prefixes in sorted order for fast lookup.

6 Dictionary and Spell Checker:
- Used to store words in sorted order for quick search, insertion and deletion.

7 Range Queries:
- AVL Trees support efficient range searches (e.g., all keys between K_1 and K_2).

8 Real-time Systems:
- AVL Trees provide guaranteed $O(\log n)$ time for operations. Hence they are useful in systems where predictability is important.

Conclusion:

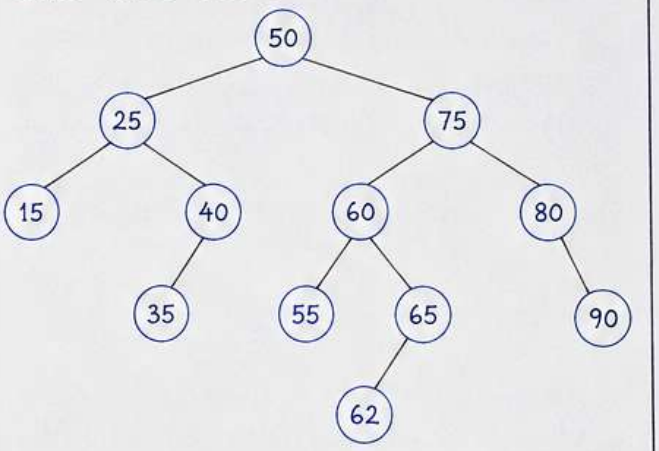
AVL Trees maintain balance using the four types of rotations (LL, RR, LR, RL). These rotations ensure that the tree remains height-balanced, providing efficient operations. AVL Trees are widely used in real-world applications where fast and reliable performance is required.

3. Explain the deletion procedure in AVL tree. Show the balancing operations after deletion of 15 from the given AVL tree. (10 Marks)

AVL Tree Deletion Procedure :

1. Perform normal BST deletion to remove the node to be deleted.
2. Starting from the parent of the deleted node, update the height of each ancestor.
3. For each ancestor node, calculate the Balance Factor (BF = Height(Left) – Height(Right)).
4. If BF = −1, 0 or +1 → the node is balanced. Move to its parent.
5. If BF = +2 or −2 → the node is unbalanced. Apply one of the four rotations (LL, RR, LR, RL) to rebalance the tree.
6. Continue this process until the root is reached or the tree becomes balanced.

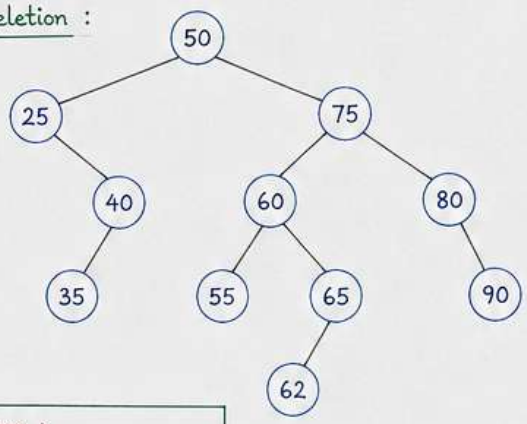
Given AVL Tree :



Step 1 : Deletion of 15

15 is a leaf node. So, simply remove it.

After deletion :



Observation :
Node 25 has BF = −2 → unbalanced.
Its Right child (40) has BF = +1.
This is a Right-Left (RL) Case.
So, perform RL rotation at node 25.

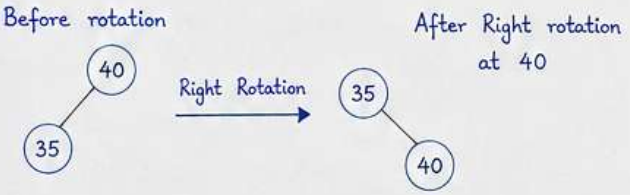
Step 2 : Update heights and balance factors

Start from the parent of deleted node (25) and move upwards.

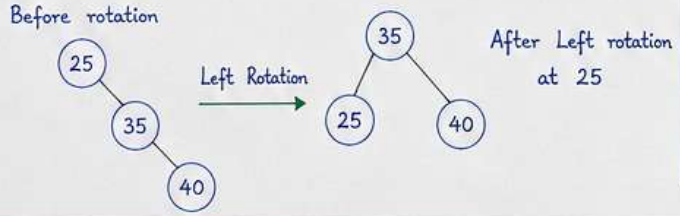
Node	Height(Left)	Height(Right)	BF = L − R
25	0	2	−2 (Unbalanced)
40	1	0	+1 (Balanced)
50	3	3	0 (Balanced)
75	3	2	+1 (Balanced)
60	1	2	−1 (Balanced)
80	0	1	−1 (Balanced)
65	0	1	−1 (Balanced)
35,55,62,90	0	0	0 (Balanced)

Step 3 : Balancing operations (RL Rotation at 25)

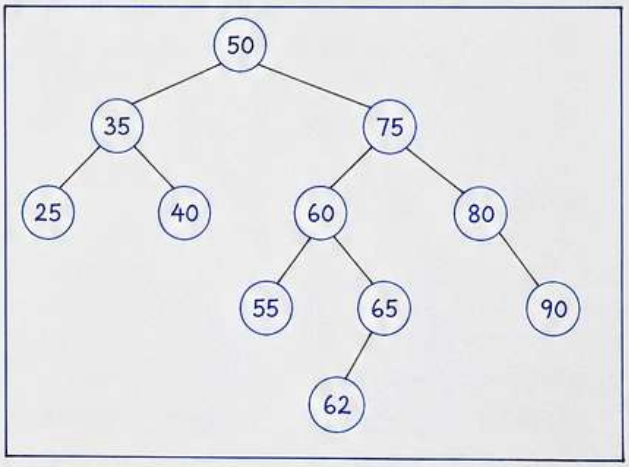
Step 3.1 : Right rotation at 40



Step 3.2 : Left rotation at 25



Step 4 : Final Balanced AVL Tree



Final Balance Factors
All nodes have BF = −1, 0 or +1.
Hence the tree is a valid AVL tree.

Conclusion : After deletion of 15, node 25 became unbalanced (BF = −2). Since its right child 40 had BF = +1, an RL rotation was performed (Right rotation at 40 followed by Left rotation at 25). The resulting tree is balanced AVL tree.

4. Construct a B-Tree of order 4 by inserting the following keys :

5, 3, 21, 9, 1, 13, 2, 7, 10, 12, 4, 8.

(10 Marks)

B-TREE OF ORDER 4

- Maximum children of a node = 4
- Maximum keys in a node = 3
- Minimum degree (t) = 2
- Minimum keys in a non-root node = t-1 = 1
- Minimum children in a non-root node = t = 2
- Keys in each node are sorted.
- Every node except root must have between 1 and 3 keys.

Insertion Algorithm (Summary)

1. If the root is full (3 keys), split it.
2. Insert the new key in the appropriate leaf.
3. If a node overflows (has 4 keys), split it and push the middle key up.
4. Repeat till the tree remains valid.

① Insert 5 :



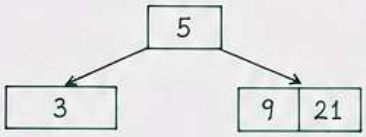
② Insert 3 :



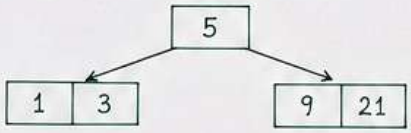
③ Insert 21 :



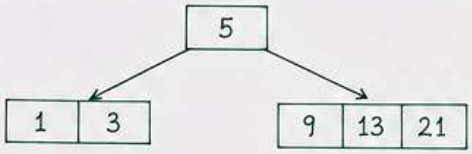
④ Insert 9 : Overflow (4 keys). Split the root. Middle key 5 goes up.



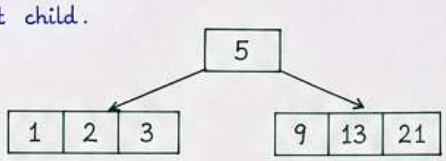
⑤ Insert 1 : Goes to left child.



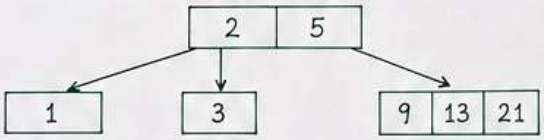
⑥ Insert 13 : Goes to right child.



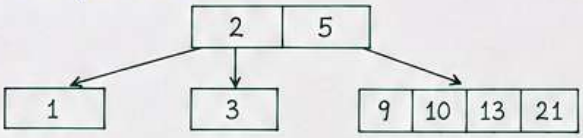
⑦ Insert 2 : Goes to left child.



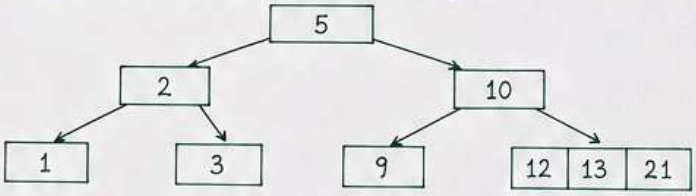
⑧ Insert 7 : Overflow in left child (4 keys). Split and promote middle key 2.



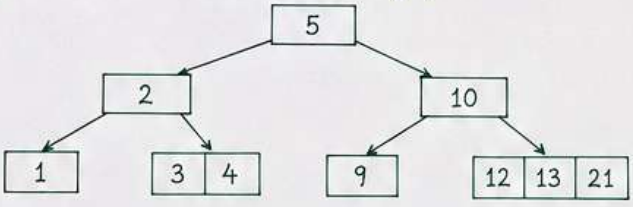
⑨ Insert 10 : Goes to rightmost child.



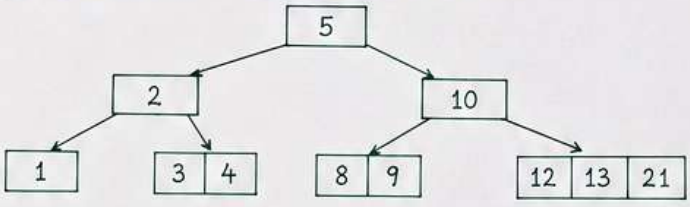
⑩ Insert 12 : Overflow in rightmost child. Split and promote middle key 10. Root also overflows. Split root and promote middle key 5.



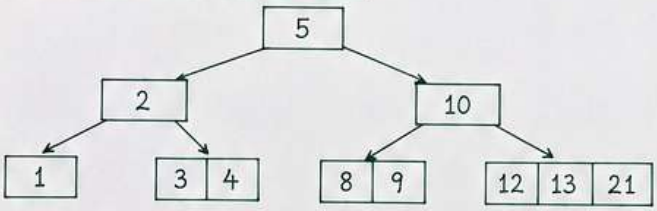
⑪ Insert 4 : Goes to node [3].



⑫ Insert 8 : Goes to node [9].



Final B-Tree of Order 4



Properties Satisfied :

1. Each node has at most 3 keys.
2. Each non-root node has at least 1 key.
3. Keys in each node are sorted.
4. All leaves are at the same level.

5. Explain the deletion operation in a B-Tree with a suitable example, and applications of B-Trees.

B-Tree : A B-Tree of order m is a balanced multi-way search tree in which

1. Each node can have at most m children.
2. Each node (except root) must have at least $\lceil \frac{m}{2} \rceil$ children.
3. Each node can contain at most m-1 keys and at least $\lceil \frac{m}{2} \rceil - 1$ keys (except root).
4. All leaves appear in the same level.

Let us consider B-Tree of order m = 4 (max. children = 4, min. children = 2, max. keys = 3, min. keys = 1).

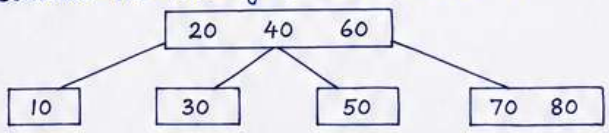
Deletion Operation in B-Tree :

To delete a key K from a B-Tree :

1. If K is present in a leaf node, delete it.
2. If K is present in an internal node:
   a) Replace it with its inorder predecessor (largest key in left subtree) or inorder successor (smallest key in right subtree) and then delete that key from the leaf where it resides.
3. After deletion, if any node has less than the minimum number of keys (i.e., less than $\lceil \frac{m}{2} \rceil - 1$ keys), then fix it by:
   a) Borrow (Redistribute) a key from its adjacent sibling having more than minimum keys.
   b) Merge (Coalesce) with a sibling when no sibling can lend a key.
   c) If the root has 0 keys after merging, make its only child as new root.

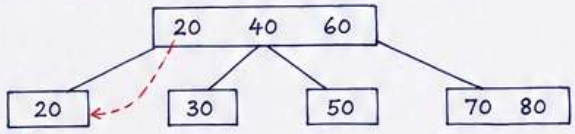
Example :

Consider the following B-Tree of order 4.



Step 1: Delete 10 (leaf node)

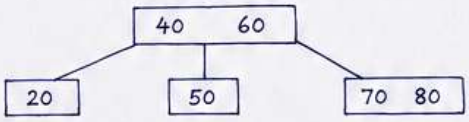
10 is in leaf. After deletion node becomes empty (< 1 key). Borrow from right sibling [30].



Result: 10 is deleted by borrowing from sibling.

Step 2: Delete 30

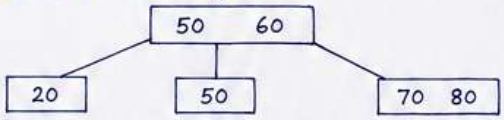
30 is in leaf. After deletion node becomes empty. Cannot borrow (both siblings have only 1 key). So merge with left sibling [20].



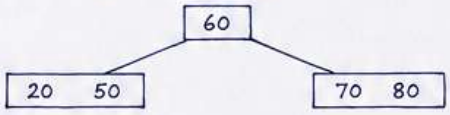
Result: Nodes [20] and [] merged and parent key 20 moved down.

Step 3: Delete 40

40 is in internal node. Replace it with its inorder successor (smallest in right subtree) which is 50. Now delete 50 from leaf.



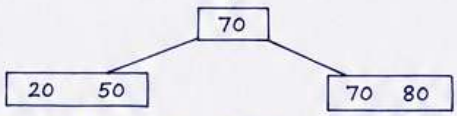
Delete 50 from leaf. Leaf becomes empty. Cannot borrow (siblings have only 1 key). Merge with left sibling [20].



Result: 50 deleted.

Step 4: Delete 60

60 is in root (internal). Replace with successor (smallest in right subtree) = 70. Now delete 70 from leaf.

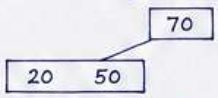


Delete 70 from leaf. Leaf becomes [80] (valid).

Result: 60 deleted.

Step 5: Delete 80

80 is in leaf. After deletion, leaf becomes empty. Cannot borrow. Merge with left sibling.



Result: 80 deleted. Final B-Tree.

Applications of B-Trees :

1. Used in DBMS and File Systems for indexing large files on disk.
2. Efficient for systems where data is stored in blocks (Disk I/O reduction).
3. Used in databases (e.g., Oracle, MySQL – InnoDB uses B+ Trees).
4. Used in operating systems for directory structures.
5. Used in range searching, sorting, and multi-level indexing.

6. Construct a Min/Max Heap by inserting the following elements:
40, 15, 30, 10, 12, 6, 25, 18.

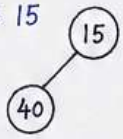
MIN HEAP (Insert one by one)

Step 1: Insert 40



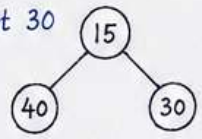
Array: [40]

Step 2: Insert 15



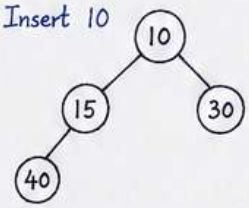
Array: [15, 40]

Step 3: Insert 30



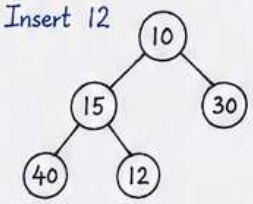
Array: [15, 40, 30]

Step 4: Insert 10



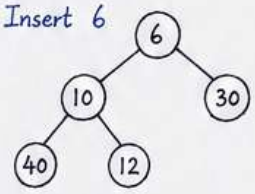
Array: [10, 15, 30, 40]

Step 5: Insert 12



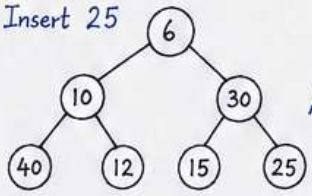
Array: [10, 15, 30, 40, 12]

Step 6: Insert 6



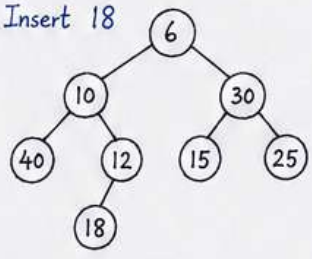
Array: [6, 10, 30, 40, 12, 15]

Step 7: Insert 25



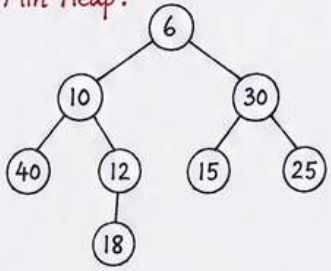
Array: [6, 10, 30, 40, 12, 15, 25]

Step 8: Insert 18



Array:
[6, 10, 30, 40, 12, 15, 25, 18]

Final Min Heap:



Array:
[6, 10, 30, 40, 12, 15, 25, 18]

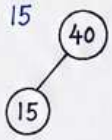
MAX HEAP (Insert one by one)

Step 1: Insert 40



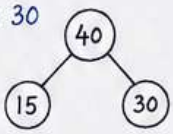
Array: [40]

Step 2: Insert 15



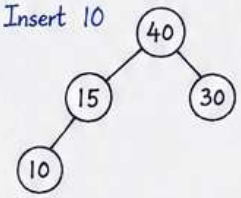
Array: [40, 15]

Step 3: Insert 30



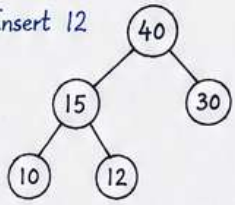
Array: [40, 15, 30]

Step 4: Insert 10



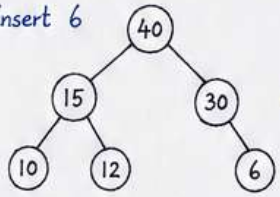
Array: [40, 15, 30, 10]

Step 5: Insert 12



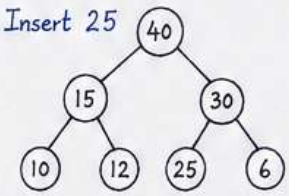
Array: [40, 15, 30, 10, 12]

Step 6: Insert 6



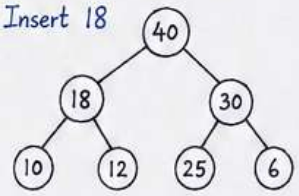
Array: [40, 15, 30, 10, 12, 6]

Step 7: Insert 25



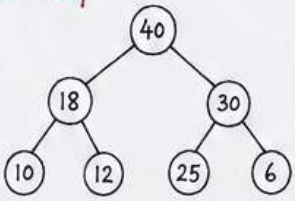
Array:
[40, 15, 30, 10, 12, 25, 6]

Step 8: Insert 18



Array:
[40, 18, 30, 10, 12, 25, 6, 15]

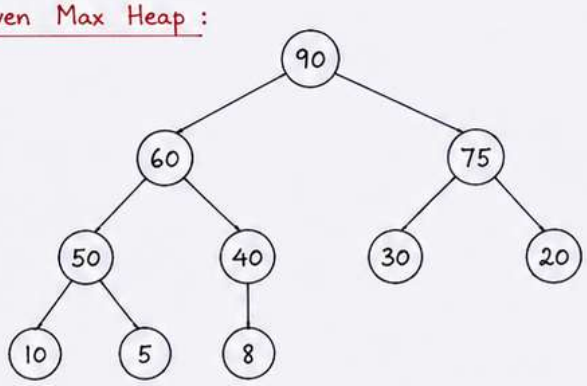
Final Max Heap:



Array:
[40, 18, 30, 10, 12, 25, 6, 15]

7. Perform one deletion operation on given Max Heap and show the step-by-step process for final Max Heap.

Given Max Heap :

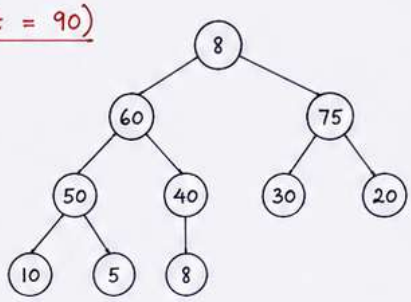


Array representation (Level order) :

1	2	3	4	5	6	7	8	9
90	60	75	50	40	30	20	10	5
								8

Deletion Operation (Delete maximum element = 90)

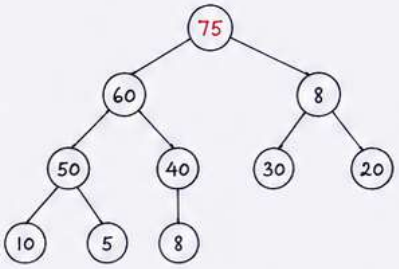
Step 1 : Delete root element (90).
Replace it with last element (8).
Remove last element.



Array :

1	2	3	4	5	6	7	8
8	60	75	50	40	30	20	10
							5

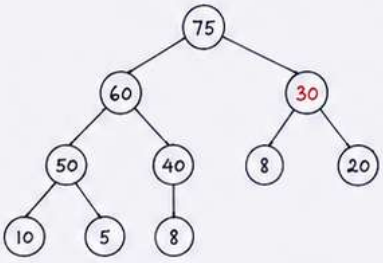
Step 2 : Heapify (sift down) the root (8).
Compare 8 with its children (60 and 75).
Swap with larger child (75).



Array :

1	2	3	4	5	6	8
75	60	8	50	40	30	20 | 10
						5

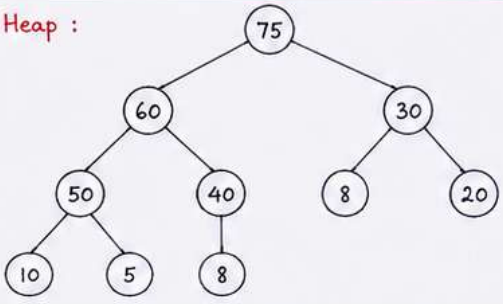
Step 3 : Now heapify at index 3 (8).
Compare 8 with its children (30 and 20).
Swap with larger child (30).



Array :

1	2	3	4	5	6	7	8
75	60	30	50	40	8	20	10
							5

Final Max Heap :



Final Array (Level order) :

1	2	3	4	5	6	7	8
75	60	30	50	40	8	20	10
							5

Maximum element (90) is deleted.
This is the final Max Heap.

8. Explain the basic operations of a Binary Heap with their time complexities and also discuss the applications of Priority Queues.

Binary Heap :

A Binary Heap is a complete binary tree that satisfies the heap property.

- **Max Heap :** The value in each node is greater than or equal to the values in its children.
- **Min Heap :** The value in each node is less than or equal to the values in its children.

A heap is usually implemented using an array. For a node at index i :

$Parent(i) = \left[\frac{i-1}{2}\right]$, $Left(i) = 2i+1$, $Right(i) = 2i+2$.

Basic Operations of Binary Heap :

Operation	Description	Time Complexity
1. Create / Make Heap	Create an empty heap. Initialize an empty array.	$O(1)$
2. Insert(x)	Insert element x at the end of the heap (next available position) and then percolate it up (heapify-up) to restore heap property.	$O(\log n)$
3. Get Maximum / Get Minimum (Peek)	Return the root element. (Max in Max-Heap, Min in Min-Heap) No change in heap.	$O(1)$
4. Delete Maximum / Delete Minimum	Remove the root element. Replace it with the last element, reduce heap size by 1 and then percolate down (heapify-down) to restore heap property.	$O(\log n)$
5. Heapify / Build Heap	Given an array of n elements, build a heap by applying heapify-down from last non-leaf node up to the root. (Bottom-up heap construction)	$O(n)$
6. Increase Key (for Max-Heap) / Decrease Key (for Min-Heap)	Increase the value of a node (Max-Heap) or decrease the value (Min-Heap) and then percolate up to restore heap property.	$O(\log n)$
7. Decrease Key (for Max-Heap) / Increase Key (for Min-Heap)	Decrease the value of a node (Max-Heap) or increase the value (Min-Heap) and then percolate down to restore heap property.	$O(\log n)$

Applications of Priority Queues :

1. **CPU Scheduling :** Used in scheduling algorithms (e.g., Highest Priority First). Tasks with higher priority are executed first.
2. **Dijkstra's Shortest Path Algorithm :** A priority queue is used to select the vertex with minimum tentative distance.
3. **Huffman Coding :** Building Huffman tree uses a priority queue to repeatedly merge two least frequent nodes.
4. **Heap Sort :** Uses a heap to repeatedly remove the maximum (or minimum) element to produce a sorted sequence.
5. **Event Simulation :** In discrete event simulations, events are processed in order of their occurrence time using a priority queue.
6. **Merging K Sorted Lists :** Used to efficiently merge multiple sorted lists (or files).
7. **Network Routing / Bandwidth Allocation :** To process packets or allocate resources based on priority.
8. **Graph Algorithms :** Prim's Minimum Spanning Tree algorithm uses a priority queue.
9. **External Sorting :** Used in multi-way merge of sorted runs in external memory.